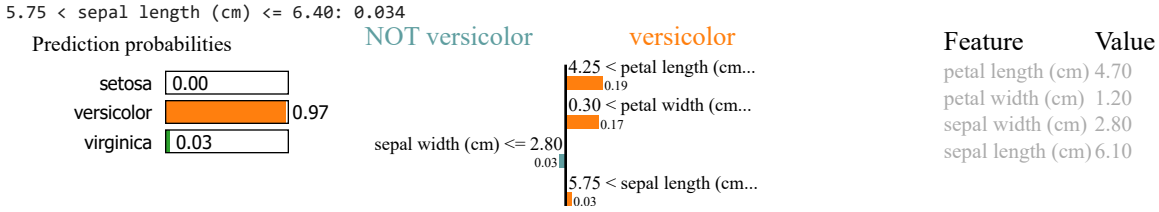
```
!pip install lime scikit-learn scikit-image matplotlib tensorflow
```

Show hidden output

```
import numpy as np
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from lime.lime_tabular import LimeTabularExplainer

# Load dataset
iris = load_iris()
X = iris.data
y = iris.target

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Train model
model = RandomForestClassifier()
model.fit(X_train, y_train)

# Create LIME explainer
explainer = LimeTabularExplainer(
    X_train,
    feature_names=iris.feature_names,
    class_names=iris.target_names,
    mode='classification'
)

# Select a sample
i = 0

# Explain prediction
exp = explainer.explain_instance(
    X_test[i],
    model.predict_proba
)

# Print explanation
print("Explanation:")
for feature, weight in exp.as_list():
    print(f"{feature}: {weight:.3f}")

# Show visualization
exp.show_in_notebook(show_table=True)
```

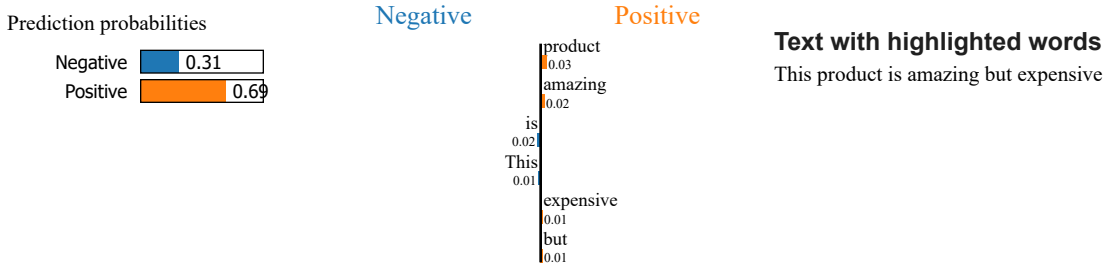
```
Explanation:
4.25 < petal length (cm) <= 5.10: 0.195
0.30 < petal width (cm) <= 1.30: 0.174
sepal width (cm) <= 2.80: -0.034
5.75 < sepal length (cm) <= 6.40: 0.034
```




```
from sklearn.pipeline import make_pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from lime.lime_text import LimeTextExplainer

# Sample dataset
texts = [
    "I love this product",
    "This is terrible",
    "Amazing experience",
    "Worst service ever",
    "I really enjoyed this",
    "I hate this so much",
```

```
    "Product is Productive",
    "Very Cheap but not expensive"

]

labels = [1, 0, 1, 0, 1, 0, 1, 1]

# Create model pipeline
model = make_pipeline(
    TfidfVectorizer(),
    LogisticRegression()
)

# Train model
model.fit(texts, labels)

# Create explainer
explainer = LimeTextExplainer(class_names=["Negative", "Positive"]) #Neutral Check with this Explainer or Label!

# Test sentence
test_text = "This product is amazing but expensive"

# Explain prediction
exp = explainer.explain_instance(
    test_text,
    model.predict_proba,
    num_features=6
)

# Print explanation
print("Explanation:")
for word, weight in exp.as_list():
    print(f"{word}: {weight:.3f}")

# Show visualization
exp.show_in_notebook()
```

```
Explanation:
product: 0.034
amazing: 0.021
is: -0.020
This: -0.014
expensive: 0.011
but: 0.011
```




```
!pip install lime scikit-image tensorflow matplotlib
```

Show hidden output

```
from google.colab import files
uploaded = files.upload()
```

Choose Files dog.jpg

dog.jpg(image/jpeg) - 3855 bytes, last modified: 3/20/2026 - 100% done

```
Saving dog.jpg to dog.jpg
```

```
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.applications.resnet50 import ResNet50, preprocess_input, decode_predictions
from tensorflow.keras.preprocessing import image
from lime import lime_image
from skimage.segmentation import mark_boundaries
import os

# Load pretrained deep model
model = ResNet50(weights='imagenet')
```

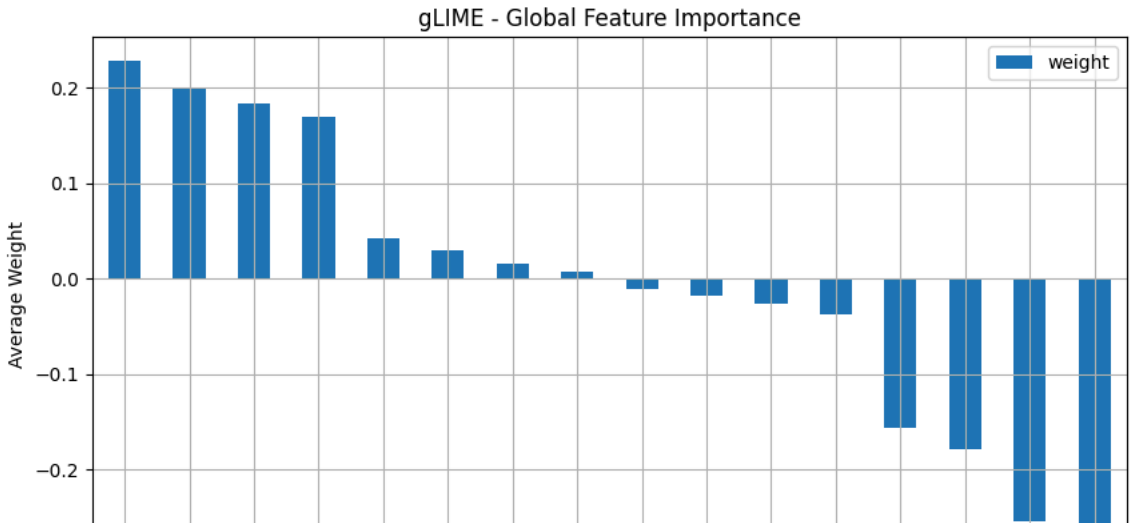
```
# Get uploaded file
img_path = list(uploaded.keys())[0]

# Load image
img = image.load_img(img_path, target_size=(224, 224))
img_array = image.img_to_array(img)

# Expand dimensions
img_batch = np.expand_dims(img_array, axis=0)
img_batch = preprocess_input(img_batch)

# Predict class
preds = model.predict(img_batch)
print("Prediction:", decode_predictions(preds, top=3)[0])

# DLIME Explainer
explainer = lime_image.LimeImageExplainer()

# Explain prediction
explanation = explainer.explain_instance(
    img_array.astype('double'),
    model.predict,
    top_labels=3,
    hide_color=0,
    num_samples=1000
)

# Get important regions
temp, mask = explanation.get_image_and_mask(
    explanation.top_labels[0],
    positive_only=True,
    num_features=5,
    hide_rest=False
)

# Show explanation
plt.figure(figsize=(6,6))
plt.imshow(mark_boundaries(temp / 255.0, mask))
plt.title("DLIME Explanation (Important Regions)")
plt.axis('off')
plt.show()
```

```
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/resnet/resnet50_weights_tf_dim_ordering_tf_
102967424/102967424 ━━━━━━━━━━━━━━━━━━━━ 1s 0us/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step
Downloading data from https://storage.googleapis.com/download.tensorflow.org/data/imagenet_class_index.json
35363/35363 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Prediction: [('n02099601', 'golden_retriever', np.float32(0.8869766)), ('n02091831', 'Saluki', np.float32(0.040618412)), ('n026
100%                                    1000/1000 [03:36<00:00,  6.24it/s]
1/1 ━━━━━━━━━━━━━━━━━━━━ 4s 4s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 3s 3s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
```

```
!pip install lime scikit-learn pandas matplotlib
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
Collecting lime
  Downloading lime-0.2.0.1.tar.gz (275 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 275.7/275.7 kB 8.2 MB/s eta 0:00:00
  Preparing metadata (setup.py) ... done
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1.6.1)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.2.2)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from lime) (2.0.2)
Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages (from lime) (1.16.3)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from lime) (4.67.3)
Requirement already satisfied: scikit-image>=0.12 in /usr/local/lib/python3.12/dist-packages (from lime) (0.25.2)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (3.6.0)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.3)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (4.62.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.5.0)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (26.0)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (3.3.2)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas) (1.17.0
Requirement already satisfied: networkx>=3.0 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime) (3.6.1)
Requirement already satisfied: imageio!=2.35.0,>=2.33 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime)
```

```
Requirement already satisfied: tifffile>=2022.8.12 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime) (2
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step
Requirement already satisfied: lazy-loader>=0.4 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime) (0.5)
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
Building wheels for collected packages: lime
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
  Building wheel for lime (setup.py) ... done
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
  Created wheel for lime: filename=lime-0.2.0.1-py3-none-any.whl size=283834 sha256=314c06a71134d82070cfe5d9af789dcc0806d536dfd2
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step
  Stored in directory: /root/.cache/pip/wheels/e7/5d/0e/4b4fff9a47468fed5633211fb3b76d1db43fe806a17fb7486a
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step
Successfully built lime
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
Installing collected packages: lime
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
Successfully installed lime-0.2.0.1
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from lime.lime_tabular import LimeTabularExplainer

# Load dataset
iris = load_iris()
X = iris.data
y = iris.target

feature_names = iris.feature_names
class_names = iris.target_names

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Train model
model = RandomForestClassifier()
model.fit(X_train, y_train)

# Create LIME explainer
explainer = LimeTabularExplainer(
    X_train,
    feature_names=feature_names,
    class_names=class_names,
    mode='classification'
)

# 🔥 gLIME: Collect explanations
all_explanations = []

num_samples = 30  # number of points to explain

for i in range(num_samples):
    exp = explainer.explain_instance(
        X_test[i],
        model.predict_proba
    )

    for feature, weight in exp.as_list():
        all_explanations.append((feature, weight))

# Convert to DataFrame
df = pd.DataFrame(all_explanations, columns=["feature", "weight"])

# Aggregate (GLOBAL IMPORTANCE)
global_importance = df.groupby("feature").mean().sort_values(by="weight", ascending=False)

print("🌍 Global Feature Importance:\n")
print(global_importance)
# Plot global importance
global_importance.plot(kind='bar', figsize=(10,5))

plt.title("gLIME - Global Feature Importance")
plt.ylabel("Average Weight")
plt.xlabel("Features")
plt.xticks(rotation=45)
plt.grid()
plt.show()
```

```
Global Feature Importance:

                                     weight
feature
1.30 < petal width (cm) <= 1.80   0.228557
4.25 < petal length (cm) <= 5.10  0.198163
0.30 < petal width (cm) <= 1.30   0.183699
1.50 < petal length (cm) <= 4.25  0.169585
5.10 < sepal length (cm) <= 5.75  0.042416
3.00 < sepal width (cm) <= 3.40   0.029390
5.75 < sepal length (cm) <= 6.40  0.015927
sepal width (cm) > 3.40           0.007810
2.80 < sepal width (cm) <= 3.00  -0.011405
sepal length (cm) > 6.40         -0.017870
sepal width (cm) <= 2.80         -0.026615
sepal length (cm) <= 5.10        -0.037891
petal length (cm) <= 1.50        -0.156572
petal width (cm) > 1.80          -0.178176
petal width (cm) <= 0.30         -0.253720
petal length (cm) > 5.10         -0.260671
```